R – Ključni, a včasih težko razumljivi koncepti

Funkcije

Funkcije so v R-u orodja, ki nekaj naredijo.

Imajo vedno isto strukturo:

ime_funkcije (*argument1, argument2, argument3…*)

Argumenti povejo funkciji na čem naj dela (podatki) in opredelijo podrobnosti kako naj kaj naredi (parametri).

Spredaj in zadaj vedno (). Tudi če imamo funkcijo brez argumentov, ne bo delala brez ().

Primer od zadnjič: ls()

Funkcije

Če se malo igramo…

Imamo funkcijo **`zabij()`** in dva objekta, **„`žebelj`“** in **„`deska`“**

> žebelj

>deska

```
>zabij(žebelj, deska)
```

Funkcije

Lahko pa povemo bolj podrobno, npr. s parametrom „globina“…

```
>zabij(žebelj, deska, globina = "do_pol")
```

```
>zabij(žebelj, deska, globina = "do_konca")
```

Funkcije

Včasih gre kaj narobe… funkcija lahko ima „bug“…

```
>zabij(žebelj, deska)
```

Funkcije

…lahko pa narobe opredelimo argumente
in rezultat ni čisto to kar bi želeli.

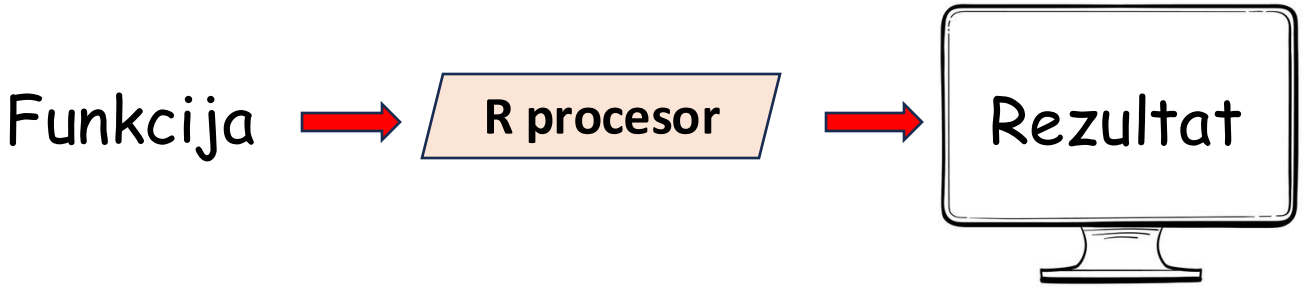
```
>zabij(žebelj, prst)
```

Funkcije

Če se vrnemo spet malo v R…

Npr. funkcija **c** nam naredi vektor:

```
> c(1, 33, 25, 68)
[1]  1 33 25 68
```



Če izhoda iz funkcije ne spravimo nikamor, se izpiše na ekran (kot v tem primeru)

Funkcije

Če se vrnemo spet malo v R…

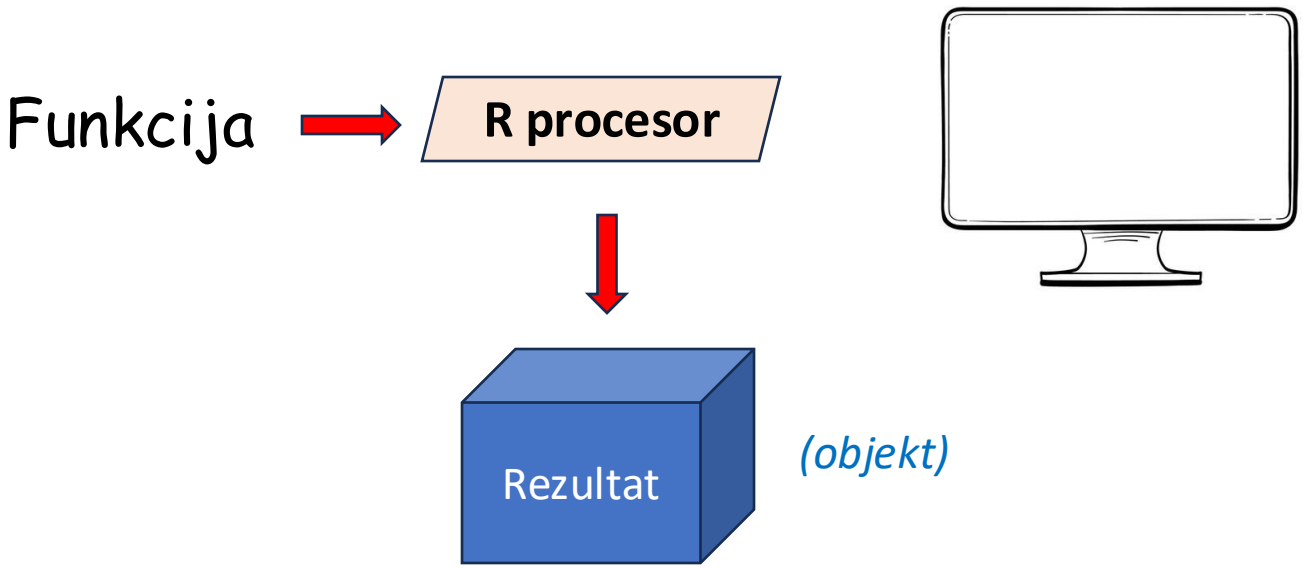
Npr. funkcija **c** nam naredi vektor:

```
> c(1, 33, 25, 68)
[1]  1 33 25 68
```

Če izhoda iz funkcije ne spravimo nikamor, se izpiše na ekran (kot v tem primeru)

Lahko pa ga shranimo:

```
test = c(1, 33, 25, 68)
```



Funkcije

Če se vrnemo spet malo v R…

Npr. funkcija **c** nam naredi vektor:

```
> c(1, 33, 25, 68)
[1]  1 33 25 68
```

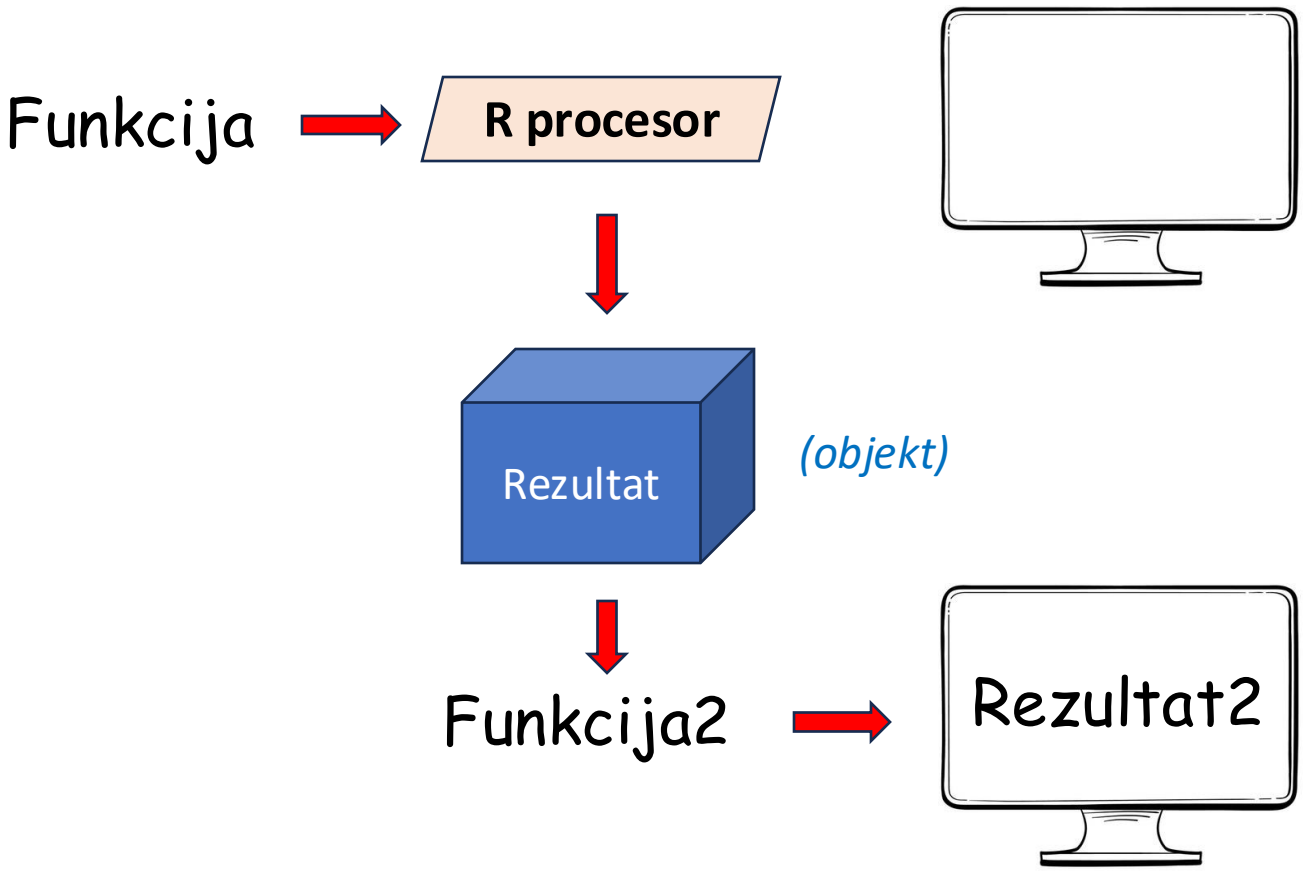
Če izhoda iz funkcije ne spravimo nikamor, se izpiše na ekran (kot v tem primeru)

Lahko pa ga shranimo:

```
test = c(1, 33, 25, 68)
```

In rezultat uporabimo v drugi funkciji.

```
 > mean(test)
[1] 31.75
```



Funkcije

Funcija ima lahko **več vhodnih argumentov**, ima pa vedno največ **en izhodni objekt** (rezultat)**.**

Funkcije lahko zelo enostavno pišemo tudi sami.

Najprej funkcijo definiramo:

```
vesela_matematika = function (a, b) {
        rezultat = paste(a+b,"Juhuhu!", sep=" ")
        return(rezultat)
}
```

Potem pa jo pokličemo z argumenti in nam da rezultat:

```
> vesela_matematika(4, 6)
[1] "10 Juhuhu!"
```

Funkcije

```
vesela_matematika = function (a, b) {
        rezultat = paste(a+b,"Juhuhu!", sep=" ")
        return(rezultat)
}
```

Funkcija se napiše vedno na isti način

ime Ukaz „function“ Argumenti (postanejo **objekti** znotraj funkcije)

```
ime_funkcije = function (argument1, argument2, …) {
        ukaz1
        ukaz2
        ukaz3
        return(rezultat)
}
```

{ = začetek kode

„telo“ funkcije – tu napišemo ukaze, ki se izvedejo ob klicu

Ukaz „return“ vrne **končni rezultat**

} = konec kode

Zanke (loop)

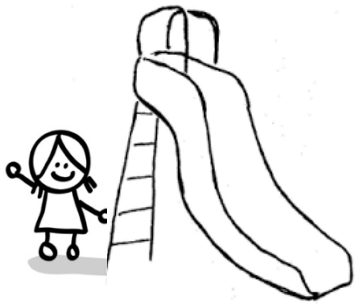
Pogosto želimo isto analizo/funkcijo pognati večkrat.

Ilustrirajmo. Napišemo funkcijo za spust po toboganu:

```
tobogan = function (otrok) {
            splezaj (otrok)
            pelji_se(otrok)
}


tobogan ("Metka")
```

```
splezaj (“Metka”)
pelji_se(“Metka”)
```



Zanke (loop)

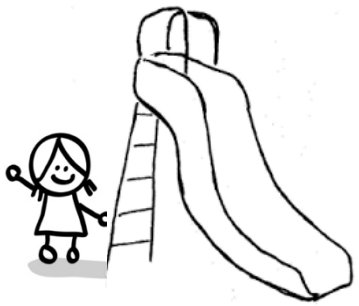
Najbolj pogosta zanka je „for“.

Recimo, da se Metka želi spustiti po toboganu trikrat.

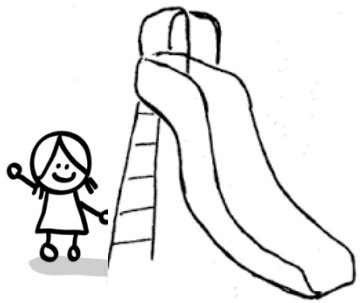
```
for (i in 1:3) {
    tobogan ("Metka")
}
```

'i' gre po vseh vrednostih vektorja 1:3

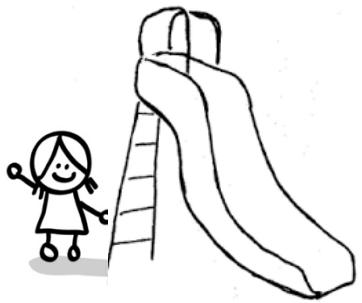
vse znotraj {} se izvede za vsako vrednost i



i = 1



i = 2



i = 3

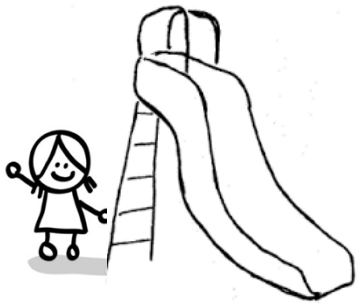
Zanke (loop)

Lahko pa imamo vektor otrok, ki bi se vozili po toboganu.

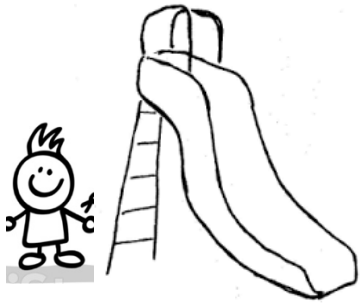
```
otroci = c ("Metka", "Miha", "Živa", "Leon")

for (i in otroci) {
    tobogan (i)
}
```

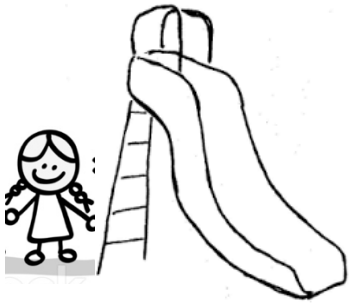
i gre skozi vse vrednosti vektorja ‚otroci' in pri vsakem prehodu vzame drugo vrednost



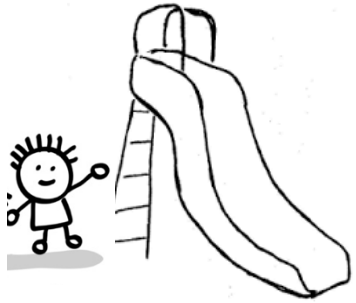
i = "Metka"



i = "Miha"



i = "Živa"



i = "Leon"

Podatkovni tipi v R-u

Primitivni (atomarni) tipi podatkov v R-u:

- **Numerični** (cela števila (integer), decimalna števila (double), kompleksna števila)
- **Besedilo** (character)
- **Logične vrednosti** (TRUE, FALSE)

Iz teh lahko delamo **vektorje**, **matrike** in **liste**.

Podatkovni tipi v R-u

```
> a = 49
> sqrt(a)
[1] 7
```

numeric (številke)

```
> a = "Kuža je pojedel domačo nalogo"
> sub("Kuža", "Muc",a)
[1] "Muc je pojedel domačo nalogo"
```

character string (besedilo)

```
> a = (1+1==3)
> a
[1] FALSE
```

Logical (logične vrednosti)

Opomba: tale znak > pomeni samo, da smo to mi napisali kot ukaz

Vektorji

Enodimenzionalne podatkovne strukture.

```
> a = c(1,2,3)
> a*2
[1] 2 4 6
```

Naredili smo vektor ‚a‘ in ga množili z 2, dobili nov vektor.

```
> a = c("muc", "pes", "polž")
> paste(a, "ek", sep="")
[1] "mucek"  "pesek"  "polžek„
```

Besedilo gre VEDNO V NAREKOVAJIH!

```
> a = c(1,2,3)
> a == 2
[1] FALSE  TRUE FALSE
```

Rezultat logičnega operatorja je logični vektor.

Vektorji

```
> a=c(1, 2.5, 4, 8.2, 332, 66, 88, 0.2, -5)
```

1	**2.5**	**4**	**8.2**	**8**	**332**	**66**	**88**	**0.2**	**-5**	Vektor a
1	2	3	4	5	6	7	8	9	10	Pozicija (indeks)

Vektorji

```
> a=c(1, 2.5, 4, 8.2, 332, 66, 88, 0.2, -5)
```

1	**2.5**	**4**	**8.2**	**8**	**332**	**66**	**88**	**0.2**	**-5**
1	2	3	4	5	6	7	8	9	10

Vektor a

Pozicija (indeks)

```
> a[4]
[1] 8.2
```

4
1

8.2
1

Vektorji

```
> a=c(1, 2.5, 4, 8.2, 332, 66, 88, 0.2, -5)
```

1	2.5	4	8.2	332	66	88	0.2	-5
1	2	3	4	6	7	8	9	10

Vektor a

Pozicija (indeks)

6	7	8	9
1	2	3	4

```
> a[c(6,7,8,9)]
[1] 332.0 66.0 88.0  0.2
```

332	66	88	0.2
1	2	3	4

Vektorji

```
> a = c("muc", "pes", "polž")
```

muc	pes	polž
1	2	3

Vektor a

Pozicija (indeks)

```
> a[3]
[1] "polž"
```

polž
1

Vector -> Dataframe

```
> vrsta = c("muc", "pes", "polž")
> stevilo = c(10, 12, 8)
> ima_bolhe = c(TRUE, TRUE, FALSE)
```

Vector -> Dataframe

```
> vrsta = c("muc", "pes", "polž")
> stevilo = c(10, 12, 8)
> ima_bolhe = c(TRUE,TRUE,FALSE)
```

Vektorji

vrsta	muc	pes	polž
stevilo	10	12	8
ima_bolhe	TRUE	TRUE	FALSE

Vector -> Dataframe

```
> vrsta = c("muc", "pes", "polž")
> stevilo = c(10, 12, 8)
> ima_bolhe = c(TRUE, TRUE, FALSE)
```

Vektorji

vrsta	muc	pes	polž
stevilo	10	12	8
ima_bolhe	TRUE	TRUE	FALSE

```
> bolhe = data.frame(vrsta,stevilo,ima_bolhe)
```

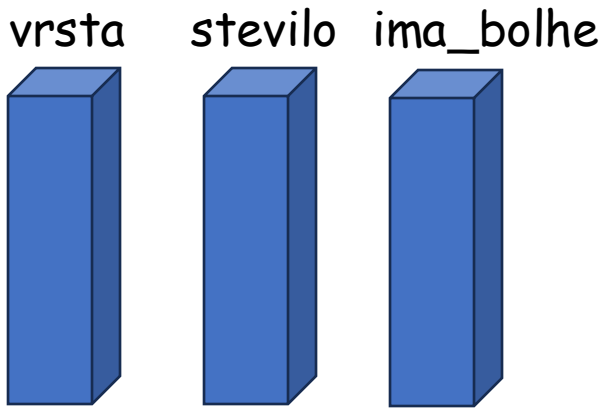
Dataframe

bolhe

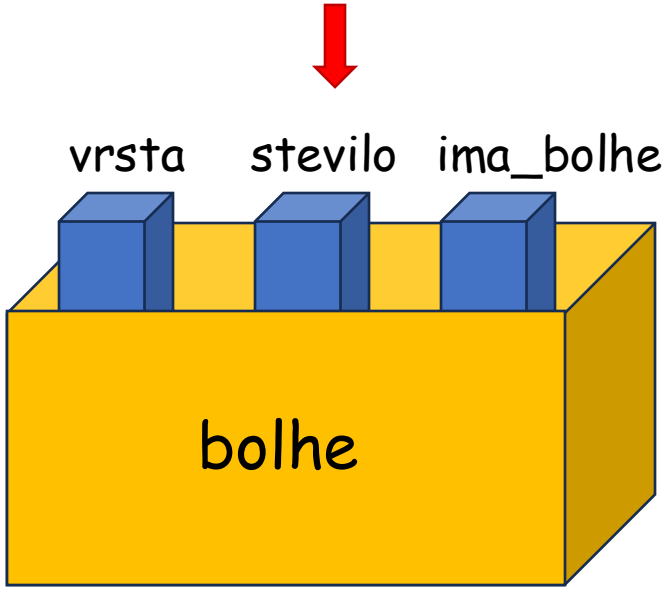
vrsta	stevilo	ima_bolhe
muc	10	TRUE
pes	12	TRUE
polž	8	FALSE

Vector -> Dataframe

```
> vrsta = c("muc", "pes", "polž")
> stevilo = c(10, 12, 8)
> ima_bolhe = c(TRUE, TRUE, FALSE)
```

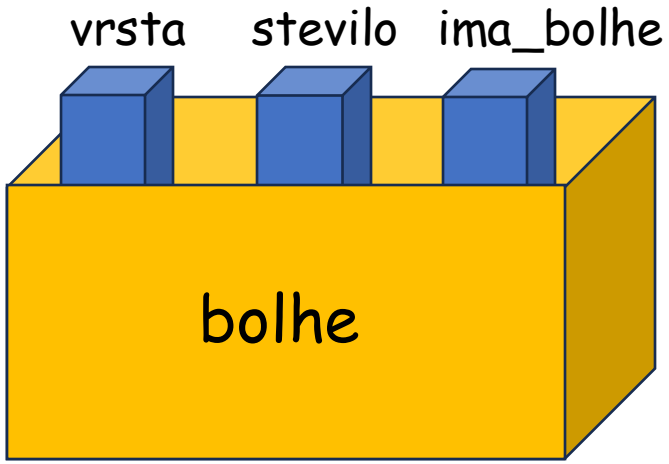



```
> bolhe = data.frame(vrsta,stevilo,ima_bolhe)
```

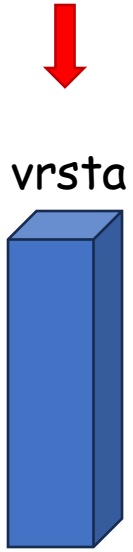


Dataframe

Vector -> Dataframe




```
> bolhe$vrsta
```



Dataframe

bolhe

vrsta	stevilo	ima_bolhe
muc	10	TRUE
pes	12	TRUE
polž	8	FALSE

← names

← data

Dataframe

bolhe

vrsta	stevilo	ima_bolhe
muc	10	TRUE
pes	12	TRUE
polž	8	FALSE

← names

← data

```
> names(bolhe)
[1] "vrsta"     "stevilo"   "ima_bolhe"
```

Vektor imen stolpcev…

Dataframe

bolhe

vrsta	stevilo	ima_bolhe
muc	10	TRUE
pes	12	TRUE
polž	8	FALSE

← names

← data

```
> names(bolhe)
[1] "vrsta"     "stevilo"   "ima_bolhe"
```

Vektor imen stolpcev…

```
> names(bolhe) = c("species","num","has_fleas")
```

… ki jih lahko zamenjamo (preimenujemo stolpce).

Dataframe

bolhe

species	num	has_fleas
muc	10	TRUE
pes	12	TRUE
polž	8	FALSE

Indeks vrstic

Indeks stolpcev

Dataframe: bolhe

	1	2	3
	vrsta	stevilo	ima_bolhe
1	muc	10	TRUE
2	pes	12	TRUE
3	polž	8	FALSE

ime_dataframa[**vrstice**, **stolpci**] *Dobimo posamezne podatke*

Indeks vrstic

Indeks stolpcev

Dataframe: bolhe

	1	2	3
	vrsta	stevilo	ima_bolhe
1	muc	10	TRUE
2	pes	12	TRUE
3	polž	8	FALSE

```
>bolhe[1, 1]
```

[1] "muc"

„Iz dataframe-a vzami prvi stolpec, prvo vrstico“

Indeks vrstic Indeks stolpcev

Dataframe: bolhe

	1	2	3
	vrsta	stevilo	ima_bolhe
1	muc	10	TRUE
2	pes	12	TRUE
3	polž	8	FALSE

```
>bolhe[1, ]
```

```
  vrsta stevilo ima_bolhe
1 "muc"      2      TRUE
```

„Iz dataframe-a vzami prvo vrstico, vse stolpce“

Indeks vrstic Indeks stolpcev

Dataframe: bolhe

	1	2	3
	vrsta	stevilo	ima_bolhe
1	muc	10	TRUE
2	pes	12	TRUE
3	polž	8	FALSE

```
>bolhe[ ,1]
```

[1] "muc" "pes" "polž"

„Iz dataframe-a vzami prvi stolpec

Indeks vrstic

Indeks stolpcev

Dataframe: bolhe

	1	2	3
	vrsta	stevilo	ima_bolhe
1	muc	10	TRUE
2	pes	12	TRUE
3	polž	8	FALSE

```
> bolhe$vrsta
```

```
[1] "muc"  "pes"  "polž"
```

Iz dataframe-a vzami stolpec “vrsta‘“

Indeks vrstic Indeks stolpcev

Dataframe: bolhe

	1	2	3
	vrsta	stevilo	ima_bolhe
1	muc	10	TRUE
2	pes	12	TRUE
3	polž	8	FALSE

```
> bolhe[ , “vrsta“]
```

[1] "muc" "pes" "polž"

Iz dataframe-a vzami stolpec ‚vrsta‘ (malo drugače)

Indeks vrstic

Indeks stolpcev

Dataframe: bolhe

	1	2	3
	vrsta	stevilo	ima_bolhe
1	muc	10	TRUE
2	pes	12	TRUE
3	polž	8	FALSE

```
> bolhe[ , c("vrsta","stevilo")]
```

```
  vrsta stevilo
1   muc       2
2   pes      12
3  polž       4
```

Iz dataframe-a vzami <u>vse</u> vrstice, stolpca ‚vrsta' in ‚stevilo'

Indeks vrstic Indeks stolpcev

Dataframe: bolhe

	1	2	3
	vrsta	stevilo	ima_bolhe
1	muc	10	TRUE
2	pes	12	TRUE
3	polž	8	FALSE

```
> bolhe[  , c(1, 2)]
```

```
  vrsta stevilo
1   muc       2
2   pes      12
3  polž       4
```

Iz dataframe-a vzami vse vrstice, stolpca 1 in 2

Logične operacije

Vrnejo logično vrednost (TRUE ali FALSE)

Indeks vrstic

Indeks stolpcev

	1	2	3
	vrsta	stevilo	ima_bolhe
1	muc	10	TRUE
2	pes	12	TRUE
3	polž	8	FALSE

FALSE
TRUE
FALSE

„Kje vse v stolpcu ‚vrsta‘ je vrednost ‚pes‘?“

```
> bolhe$vrsta == "pes"
  [1] FALSE  TRUE FALSE
```

Logične operacije

Nam **odgovarjajo na <u>vprašanja</u>**. Odgovor je vedno **TRUE** ali **FALSE**.

== enakost (ali je A enako B)

!= neenakost (ali A <u>ni</u> enako B)

> < >= <= večje, manjše, večje ali enako, manjše ali enako

! negacija (logični “not”) (!A spremeni TRUE v FALSE in obratno)

| logični ali (A <u>ali</u> B zapišemo A | B)

& logični “in hkrati” (A <u>in hkrati</u> B zapišemo A & B)

Logične operacije

Indeks vrstic Indeks stolpcev

	1	2	3	
	vrsta	stevilo	ima_bolhe	
1	muc	10	TRUE	FALSE
2	pes	12	TRUE	TRUE
3	polž	8	FALSE	FALSE

„Samo vrstice kjer je v stolpcu „vrsta“ vrednost „pes“, vse stolpce“

```
> bolhe[bolhe$vrsta == "pes",]
  vrsta stevilo ima_bolhe
2   pes      12      TRUE
```

pes	12	TRUE

Logične operacije

Indeks vrstic Indeks stolpcev

	1	2	3	
	vrsta	stevilo	ima_bolhe	
1	muc	10	TRUE	FALSE
2	pes	12	TRUE	TRUE
3	polž	8	FALSE	TRUE

„Samo vrstice kjer je v stolpcu „vrsta“ vrednost „pes“
***<u>ALI</u>** je „stevilo“ manjše od 10, vse stolpce“*

```
> bolhe[bolhe$vrsta == "pes" | bolhe$stevilo < 10,]
  vrsta stevilo ima_bolhe
2   pes      12      TRUE
3  polž       8     FALSE
```

pes	12	TRUE
polž	8	FALSE

| je logični operator „OR“. Logični „AND“ je &, neenakost je !=.

Faktorji

Če imamo v podatkih **kategorične spremenljivke**, je zelo koristno, da to jasno opredelimo.

Naredimo vektor besed:

```
> moj_wannabe_faktor = c("muha", "muha", "miš", "muha", "miš", "hrček")

> str(moj_wannabe_faktor)
 chr [1:6] "muha" "muha" "miš" "muha" "miš" "hrček"

> summary (moj_wannabe_faktor)
   Length     Class      Mode
        6 character character
```

moj_wannabe_faktor	muha	muha	miš	muha	miš	hrček

Faktorji

Če imamo v podatkih **kategorične spremenljivke**, je zelo koristno, da to jasno opredelimo.

Zdaj pa s funkcijo „factor“ iz tega naredimo faktor:

```
> moj_faktor = factor(moj_wannabe_faktor)

> str(moj_faktor)
 Factor w/ 3 levels "hrček","miš",..: 3 3 2 3 2 1

> summary(moj_faktor)
hrček   miš  muha
    1     2     3
```

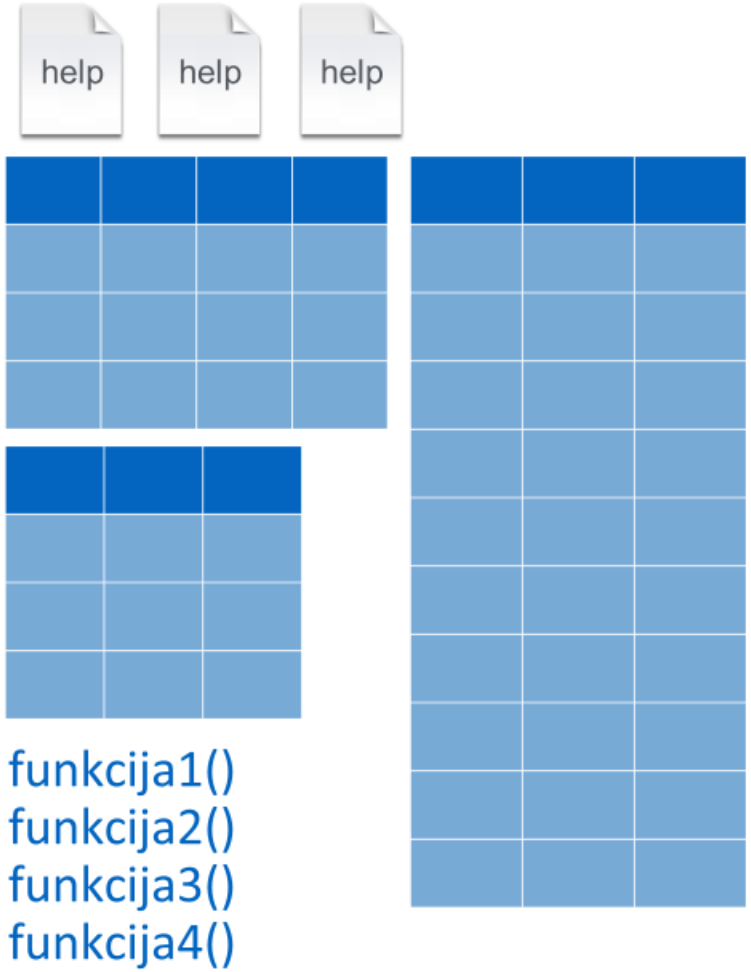
moj_faktor

3	3	2	3	2	1

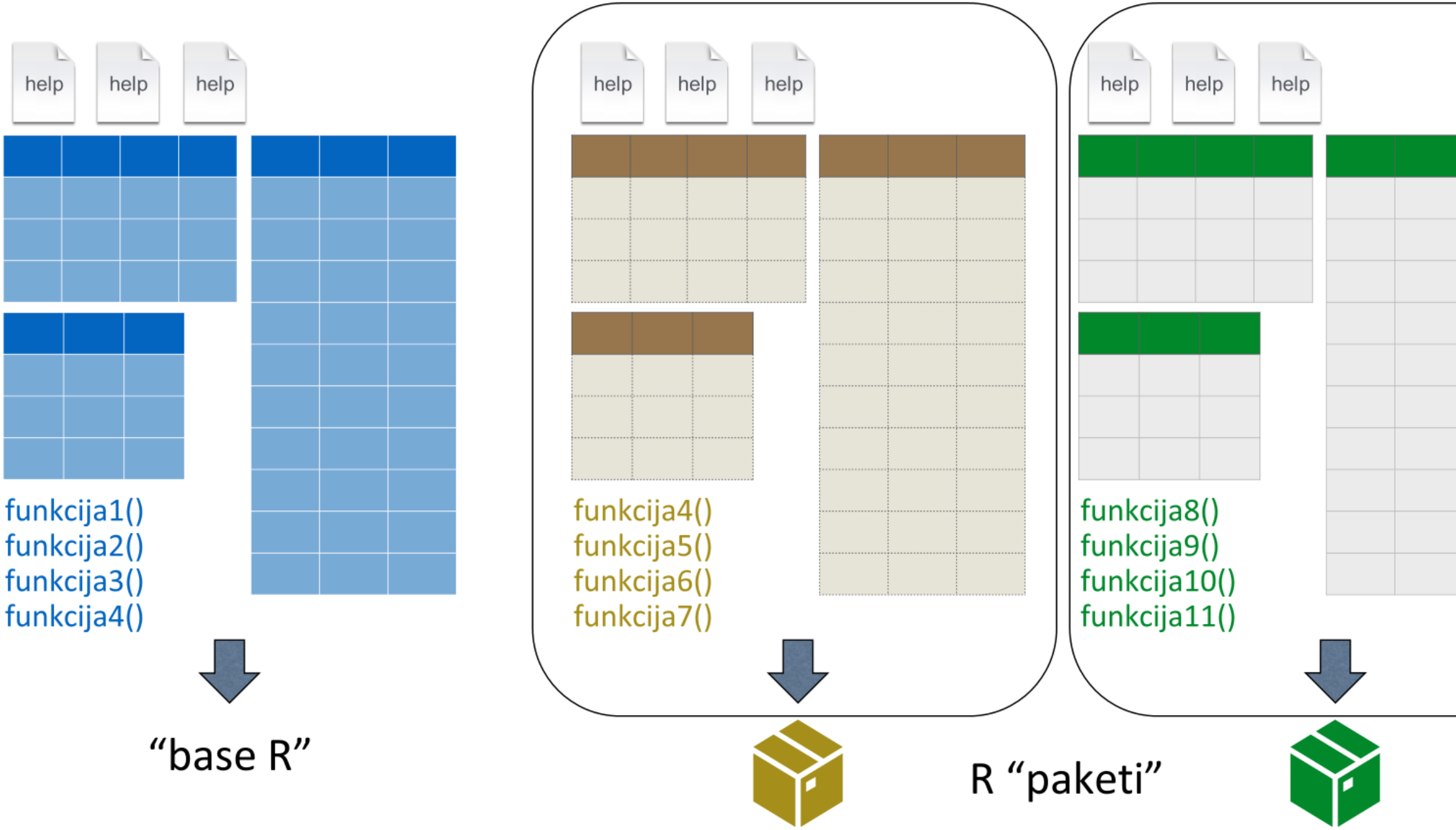
levels(moj_faktor)

1	2	3
hrček	miš	muha

R Paketi



Base R: funkcije, podatki, pravila in sistem pomoči


help
help
help
funkcija1()
funkcija2()
funkcija3()
funkcija4()
"base R"
help
help
help
funkcija4()
funkcija5()
funkcija6()
funkcija7()
help
help
help
funkcija8()
funkcija9()
funkcija10()
funkcija11()
R "paketi"

The Comprehensive R Archive

Secure https://cran.r-project.org

CRAN
Mirrors
What's new?
Task Views
Search

About R
R Homepage
The R Journal

Software
R Sources
R Binaries
Packages
Other

Documentation
Manuals
FAQs
Contributed

Available CRAN Packages By Name

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

A3	Accurate, Adaptable, and Accessible Error Metrics for Predictive Models
abbyyR	Access to Abbyy Optical Character Recognition (OCR) API
abc	Tools for Approximate Bayesian Computation (ABC)
ABCanalysis	Computed ABC Analysis
abc.data	Data Only: Tools for Approximate Bayesian Computation (ABC)
abcdeFBA	ABCDE_FBA: A-Biologist-Can-Do-Everything of Flux Balance Analysis with this package
ABCoptim	Implementation of Artificial Bee Colony (ABC) Optimization
ABCp2	Approximate Bayesian Computational Model for Estimating P2
ABC.RAP	Array Based CpG Region Analysis Pipeline
abcrf	Approximate Bayesian Computation via Random Forests
abctools	Tools for ABC Analyses
abd	The Analysis of Biological Data
abf2	Load Gap-Free Axon ABF2 Files
ABHgenotypeR	Easy Visualization of ABH Genotypes
abind	Combine Multidimensional Arrays
abjutils	Useful Tools for Jurimetrical Analysis Used by the Brazilian Jurimetrics Association
abn	Modelling Multivariate Data with Additive Bayesian Networks
abodOutlier	Angle-Based Outlier Detection

Uporaba paketov

1

```
install.packages("foo")
```

Instalira paket na računalnik

1 x za vsak računalnik

2

```
library(foo)
```

Naloži paket

1 x pri vsaki uporabi R-a